Capstone Project Report

Ashley Mawoyo
Department of Computer Science
mwyash001@myuct.ac.za

Vimbai Mutsawehuni
Department of Computer Science
mtsvim006@myuct.ac.za

Alice Makwangudze
Department of Computer Science
mkwali004@myuct.ac.za

Contents

2. Abstract 3
3. Introduction 3
4. Requirements Captured 3
4.0 Functional Requirements 3
4.1 Non-functional Requirements 4
4.2 Usability Requirements 4
4.3 Use Case Narratives: 4
4.3.1 Use Case 0 4
4.3.2 Use Case 1 4
4.3.3 Use case 2 5
4.3.4 Use Case 3 5
4.3.5 Use Case 4 5
4.3.6 Use case 5 6
4.3.7 Use Case 6 6
5. Design Overview 6
5.0 Justification of System Design 6
5.1 Layered architecture diagram 9
5.2 System Architecture and Design Philosophy 10
5.3 Approach to Algorithmic Complexity 10
6. Implementation 11
6.0 User Interface implementation 11
6.1 Most significant methods of classes 11
6.1.1 Orders 11
6.1.2 Transfers 13
6.1.3 Distributions 14
6.1.4 Delivery 15
6.1.5 Stock Take 16
6.2 Special programming techniques or libraries used 16
7. Program Validation and Verification 16

7.0 System Testing and Quality Management 16
7.1 Software Testing Plan 17
7.2 Validation of Program Correctness 17
7.3 Test Data and Results 18
7.4 Discussion of Test Results 18
8. Conclusion 19
9. AI Clause 19
Appendix A — Evidence of testing 20
Appendix B— User Manual and Code 24

2. Abstract

This project delivers a comprehensive, centralized inventory management system designed to replace an inefficient paper-based workflow. The resulting electronic system facilitates a range of core business operations, including placing and managing orders with suppliers, processing partial and full deliveries, and executing stocktakes. Furthermore, it streamlines internal logistics through features for stock distribution from a main branch to other branches and enables inter-branch transfers. By digitizing these processes, the system provides real-time data and analytics, offering actionable insights into business performance and moving away from manual record-keeping. The system is role-based, ensuring users can only access functionalities pertinent to their designated responsibilities, thereby enhancing security and operational efficiency.

3. Introduction

The primary purpose of this project is to develop and implement a robust electronic inventory management system to replace the company's existing paper-based processes. The current manual system for handling orders, deliveries, stock counts, and transfers is prone to human error, lacks real-time visibility, and presents significant administrative overhead. This project aims to address these inefficiencies by creating a centralized digital platform. The key stakeholders for this system include the **Main Branch Manager (Administrator)**, who oversees ordering and distribution; **Branch Managers**, who handle local stock and inter-branch transfers; and **Data Capture Clerks**, responsible for stocktaking and data entry.

Our approach to solving this problem involved an evolutionary prototyping model, allowing for an iterative development process where core functionalities were built and refined based on stakeholder feedback. This agile-like method ensured the final system closely aligns with the operational needs of the users. The system is designed to be a comprehensive solution, covering the entire lifecycle of inventory management. This includes:

- **Order Management**: Allowing the Main Branch Manager to place orders when stock levels are low and manage quotations.
- **Delivery Processing**: Enabling the management of both partial and fully fulfilled orders, with clear visibility into pending items.
- **Stock Distribution**: Facilitating the allocation of stock from the main branch to other branches based on sales performance and stock levels.
- **Stocktaking**: Providing tools for branch clerks to conduct physical counts and update system records accordingly.
- **Inter-Branch Transfers**: Managing and approving stock transfers between different branches to balance inventory.
- **Analytics and Reporting**: Offering a dashboard for main branch manager to view performance graphs and reports, providing actionable insights from historical data.

By consolidating these functions into a single, role-based platform, the project delivers a streamlined, efficient, and data-driven tool for managing inventory across all branches.

4. Requirements Captured

4.0 Functional Requirements

The primary functionalities of the BabyBargains system are:

Inventory Management: The system will transition from a paper-based method to a centralized electronic solution for tracking stock levels and orders across all branches.

Quotation Generation: It will generate structured JSON quotations to send to suppliers.

Inter-Branch Transfers: The system will facilitate and track the movement of inventory between different branch locations.

Delivery Tracking: It will support the tracking of partial and complete deliveries against a single quotation.

Data Validation: The system must validate and verify numerical data, such as prices and stock quantities, upon entry to ensure accuracy.

Unique Item Coding: A unique 8-digit item code will be automatically generated for every new product added to the system.

Analytics and Reporting: The system will provide comprehensive analytics and reports on the performance of all branches.

Role-Based Access Control: User access will be determined by roles, with the main branch manager having administrative privileges and other managers having more restricted access

4.1 Non-functional Requirements

Performance: While there are no strict response time requirements, the system must be efficient and user-friendly. The quotation generation process for the main branch manager should take no more than 3 minutes per transaction. Stock levels must have real-time visibility, updating automatically every 30 seconds across all branches.

Data Integrity and Validation: The system must validate all numerical data upon entry to prevent inaccurate submissions with 100% accuracy during testing. The unique 8-digit coding system must ensure no duplicates or conflicts for new items. All system-generated JSON quotations must comply with supplier formatting requirements, with a target of 99% compliance.

4.2 Usability Requirements

User Interface: The system must be user-friendly.

Role-Based Simplicity: The interface will be tailored to the user's role. Restricted functions will be hidden or disabled, ensuring users only see features relevant to their permissions.

Error Prevention: The system will prompt users to correct invalid data, such as non-existent order numbers or infeasible quantities, preventing errors before submission.

4.3 Use Case Narratives:

4.3.1 Use Case 0

Title: Log in

Actor: Main Branch Manager, Branch Manager, Clerk

The user accesses the system and enters their login credentials. The system verifies the credentials against the stored user records in the database.

If the credentials are invalid, the system displays an error message and prompts the user to re-enter valid credentials.

If the credentials are valid, the system grants access and displays the dashboard, which contains only the functionalities permitted for that user’s role.

Each role has access to a specific set of features, and restricted functions remain hidden or disabled.

4.3.2 Use Case 1

Title: Place an Order
Actor: Main Branch Manager

Pre – conditions: Stock levels should be low

Post–conditions: Supplier has confirmed that the order has been received

The admin logs into the system by entering their credentials.

The admin is then prompted to select the items they wish to order. They select the items and specify the quantities required. Where the proposed quantities are not feasible, the system requests the admin to adjust the quantities.

The system generates an Excel file containing the selected items and quantities to form a quotation. The admin reviews the quotation and can either confirm it or make changes. After confirmation, the system automatically saves the Excel file to the computer. Each generated quotation is stored in the system, allowing the admin to retrieve and modify it at any time.

4.3.3 Use case 2

Title: Manage Delivery for Partial or Fully Fulfilled Orders
Actor**:** Main Branch Manager

Preconditions: The orders have been delivered and entered into the system. Order status is recorded as partial or fully fulfilled

Post-conditions: Order delivery status and details are up to date, and pending items for partial orders are visible for follow-up

The admin logs into the system by entering their credentials. The admin selects the category of orders to view, either partially fulfilled or fully fulfilled. The system displays a filtered list of orders in the chosen category.

The admin enters or selects the order number to check. The system retrieves and displays the order details. If the order number does not exist, the system prompts the admin to enter a valid order number.

For partially fulfilled orders, the system highlights items still pending delivery. For fully fulfilled orders, the system displays a note stating, *"No pending items,"* and lists all items in the order. The admin can schedule delivery for pending items or update delivery notes/status if a partial order has now been fully fulfilled.

4.3.4 Use Case 3

Title: Stock distribution

Actors: Main branch manager

Preconditions: Stock is available at the main diagram

Postconditions: Stock distribution record is saved, relevant branch stock levels are updated

Description: The main branch manager logs in and they system verifies and authenticates. The manager selects recently delivered and available stock to be distributed, chooses recipient branches, assigns quantities and confirms distribution. The system updates stock records and send distribution note via email to the recipient branch. The manager allocates stock to branches based on sales performance, stock levels per personal discretion. If the main branch manager tries to distribute more than what is available at main branch the system blocks the distributions and warns the manager. If a branch cannot accept stock due to capacity limits the system notifies the manager and allows them to redirect stock.

4.3.5 Use Case 4

Title: Stocktaking

Actors: Data capture clerk

Preconditions: 1 month should have elapsed

Postconditions: Stock records are updated to reflect physical count

Description: The data capture clerks log in and the system verifies and authenticates. The branch clerks download products reference sheets that contains reference numbers of all products in the system. The clerk enters the reference number and selects the respective product and enter physical count into the system. The system verifies the reference number and updates the records.

4.3.6 Use case 5

Title: Track and view analytics

Actors : Main branch manager

Preconditions: Sufficient historical data exist in the database

Postconditions: The manager has actionable insights for decision making

Description: The manager logs in and the system authenticates. The manager navigates to analytics dashboard, select range or category and views performance graphs and reports. The system retrieves aggregated data from MongoDB to present trends. If there isn't sufficient data for the selected period, they system notifies the manager.

4.3.7 Use Case 6

Title: Branch Inter-Transfer

Actors: Main Branch Manager, Branch Managers

Preconditions: At least two branches exist in the system. A stock transfer request has been submitted by a branch, and another branch has sufficient stock to fulfil the request either because the requesting branch's stock levels are low or due to a special on-demand product request.

Postconditions: The request is approved, stock is reconciled between branches, and all related records are updated in the system.

Description: The main branch manager logs into the system to review and approve stock transfer requests. They select the donor and recipient branches, specify the products and quantities to be transferred, and confirm the transaction. The system deducts the specified quantities from the donor branch's inventory, adds them to the recipient branch's stock, records the transaction in a transfer log, and sends notifications to both branches via email. If the donor branch does not have sufficient stock, the system alerts the manager and prevents the transfer from being completed.

5. Design Overview

This section outlines the complete design philosophy for the BabyBargains stock management system, covering the justification for the object-oriented design, the core architectural patterns employed, and the strategic approach to algorithmic complexity.

5.0 Justification of System Design

The proposed system design, as represented by the analysis class diagram, provides a robust and logical framework that directly addresses the functional requirements for transitioning BabyBargains to a centralized electronic stock management system. The object-oriented approach effectively models the real-world entities and processes involved, ensuring that the final product will exhibit the expected behaviour in terms of efficiency, accountability, and data integrity.

1. Core Operations: Orders and Delivery Management

- **Order and Supplier Integration:** The Order and Supplier classes establish a clear relationship for tracking procurement. The Order class acts as a central hub for purchasing activities, containing methods like generateJSONQuotation(), which directly fulfils the requirement for an electronic quote generation system. This allows for easy comparison of supplier prices as mandated.

- **Decoupled and Traceable Deliveries:** A key strength of the design is the separation between an Order and a Delivery. An Order is composed of OrderItems (what was requested), while a Delivery is composed of DeliveryItems (what was received). This one-to-many relationship elegantly handles the business rule of partial deliveries, allowing the system to track outstanding items and delivery completion status accurately.
- **Status Tracking:** The inclusion of a Status attribute in both the Order and Delivery classes enables real-time tracking, providing clarity on whether an order is pending, partially fulfilled, or complete.

2. Inventory and Multi-Branch Management

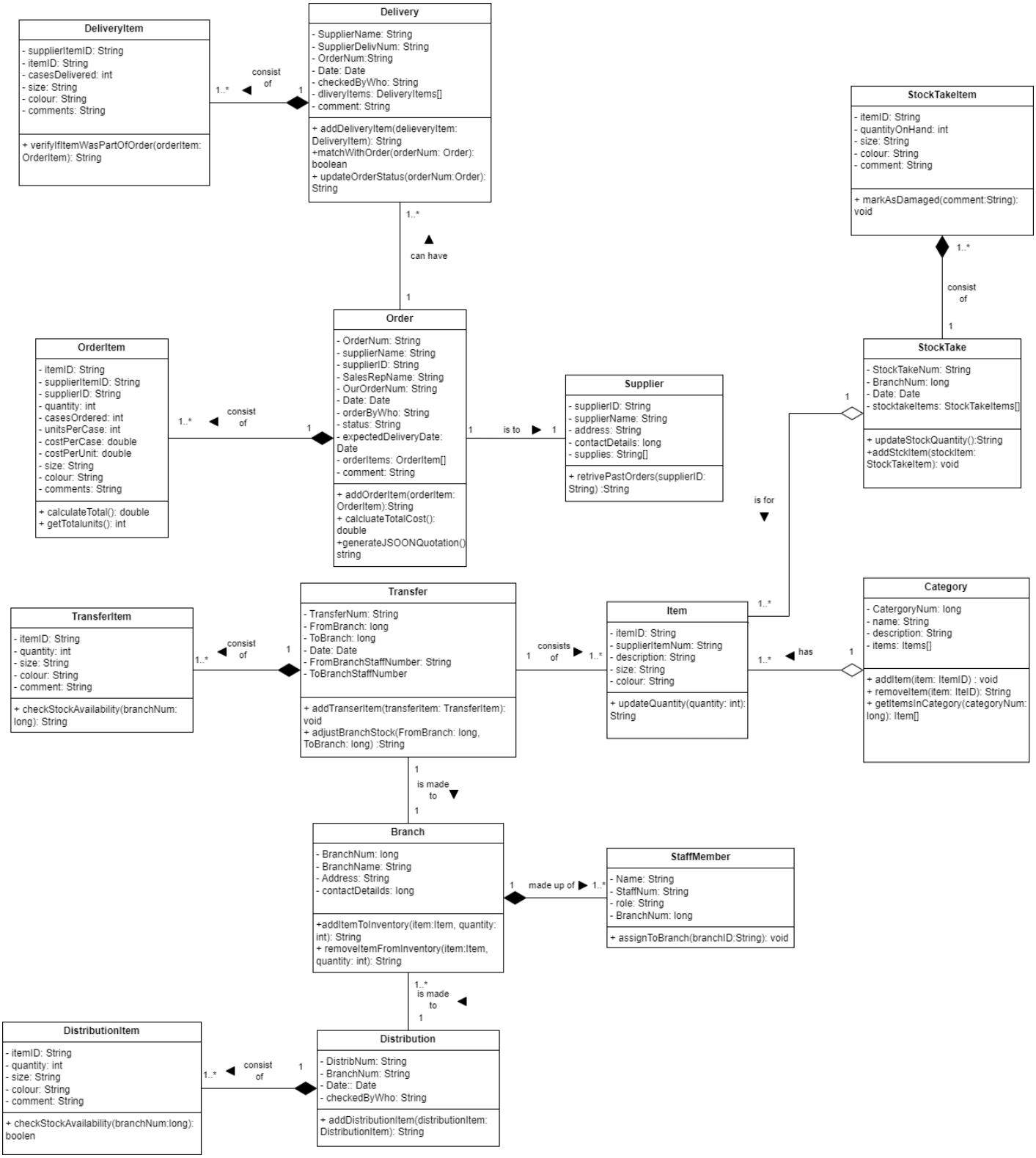
The system is designed to manage inventory across multiple branches, providing both centralized oversight and branch-level functionality as specified in the project scope.

- **Centralized Item Master:** The Item class serves as the master record for every product, with its quantity attribute providing a real-time view of stock levels. The updateQuantity() method is the designated interface for all stock modifications, ensuring that changes from sales, deliveries, or transfers are handled consistently.
- **Inter-Branch Transfer Accountability:** The requirement for transparent and accountable inter-branch transfers is explicitly modeled by the Transfer and TransferItem classes. This structure captures all critical information, including the originating and destination branches, creating a clear audit trail for all inventory movements.
- **Role-Based Access Control:** The relationship between the Branch and StaffMember classes, combined with the role attribute, provides the necessary foundation for implementing specified access privileges. This design allows the system to differentiate between the main branch manager (administrative rights) and other managers (restricted access).
- **Dedicated Stocktaking Functionality:** The StockTake and StockTakeItem classes directly support the requirement for managers to conduct periodic stocktaking, providing a mechanism to record on-hand inventory and markAsDamaged(), improving accuracy.

3. Data Integrity and Item Traceability

The design places a strong emphasis on data accuracy and traceability, which is critical for a centralized inventory system.

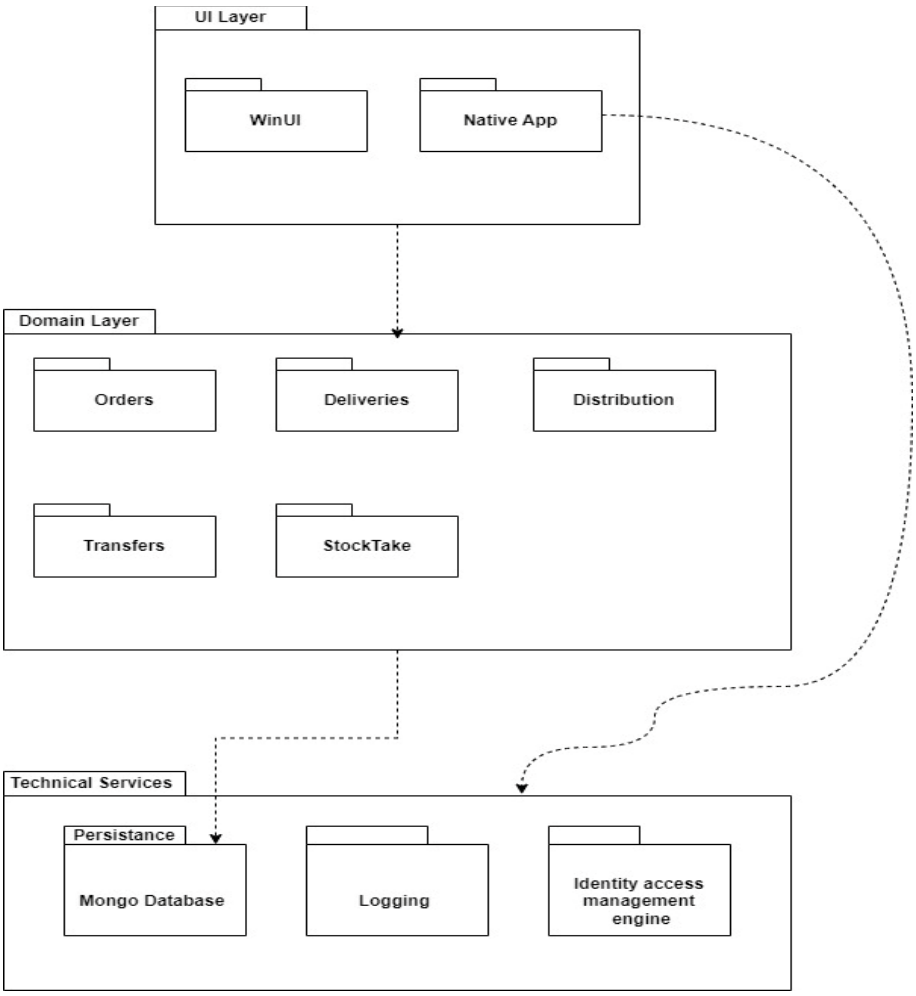
- **Unique Item Identification:** The consistent use of itemID as a primary key across all relevant classes is fundamental. This directly supports the implementation of the unique 8-digit item coding scheme, ensuring every item can be traced from its initial order to its current location and status.
- **Built-in Validation Logic:** The inclusion of specific verification methods, such as checkStockAvailability() and verifyItemWasPartOfOrder(), demonstrates that the design incorporates points for data validation. This prevents logical errors, such as transferring out-of-stock items, thereby ensuring data accuracy.


DeliveryItem
- supplierItemID: String
- itemID: String
- casesDelivered: int
- size: String
- colour: String
- comments: String
+ verifyIfItemWasPartOfOrder(orderItem: OrderItem): String
1..*
consist of
1
Delivery
- SupplierName: String
- SupplierDelivNum: String
- OrderNum:String
- Date: Date
- checkedByWho: String
- dliveryItems: DeliveryItems[]
- comment: String
+ addDeliveryItem(delieveryItem: DeliveryItem): String
+matchWithOrder(orderNum: Order): boolean
+ updateOrderStatus(orderNum:Order): String
1..*
can have
1
StockTakeItem
- itemID: String
- quantityOnHand: int
- size: String
- colour: String
- comment: String
+ markAsDamaged(comment:String): void
1..*
consist of
1
StockTake
- StockTakeNum: String
- BranchNum: long
- Date: Date
- stocktakeItems: StockTakeItems[]
+ updateStockQuantity():String
+addStckItem(stockItem: StockTakeItem): void
Order
- OrderNum: String
- supplierName: String
- supplierID: String
- SalesRepName: String
- OurOrderNum: String
- Date: Date
- orderByWho: String
- status: String
- expectedDeliveryDate: Date
- orderItems: OrderItem[]
- comment: String
+ addOrderItem(orderItem: OrderItem):String
+ calcluateTotalCost(): double
+generateJSOONQuotation() string
OrderItem
- itemID: String
- supplierItemID: String
- supplierID: String
- quantity: int
- casesOrdered: int
- unitsPerCase: int
- costPerCase: double
- costPerUnit: double
- size: String
- colour: String
- comments: String
+ calculateTotal(): double
+ getTotalunits(): int
1..*
consist of
1
1
is to
1
Supplier
- supplierID: String
- supplierName: String
- address: String
- contactDetails: long
- supplies: String[]
+ retrivePastOrders(supplierID: String) :String
1
is for
1..*
Category
- CatergoryNum: long
- name: String
- description: String
- items: Items[]
+ addItem(item: ItemID) : void
+ removeItem(item: IteID): String
+ getItemsInCategory(categoryNum: long): Item[]
Transfer
- TransferNum: String
- FromBranch: long
- ToBranch: long
- Date: Date
- FromBranchStaffNumber: String
- ToBranchStaffNumber
+ addTranserItem(transferItem: TransferItem): void
+ adjustBranchStock(FromBranch: long, ToBranch: long) :String
TransferItem
- itemID: String
- quantity: int
- size: String
- colour: String
- comment: String
+ checkStockAvailability(branchNum: long): String
1..*
consist of
1
1
consists of
1..*
Item
- itemID: String
- supplierItemNum: String
- description: String
- size: String
- colour: String
+ updateQuantity(quantity: int): String
1..*
has
1
1
is made to
1
Branch
- BranchNum: long
- BranchName: String
- Address: String
- contactDetailds: long
+addItemToInventory(item:Item, quantity: int): String
+ removeItemFromInventory(item:Item, quantity: int): String
1
made up of
1..*
StaffMember
- Name: String
- StaffNum: String
- role: String
- BranchNum: long
+ assignToBranch(branchID:String): void
1..*
is made to
1
DistributionItem
- itemID: String
- quantity: int
- size: String
- colour: String
- comment: String
+ checkStockAvailability(branchNum:long): boolen
1..*
consist of
1
Distribution
- DistribNum: String
- BranchNum: String
- Date:: Date
- checkedByWho: String
+ addDistributionItem(distributionItem: DistributionItem): String

5.1 Layered architecture diagram

The system is designed using a layered architecture, a standard practice for building maintainable and scalable applications. This structure separates the system into three distinct layers, each with a specific responsibility and a clear one-way flow of dependency.

- **UI Layer (Presentation):** This is the user-facing C# WinUI application. Its sole purpose is to present data to the user and capture their input. It contains no business logic and delegates all actions to the Domain Layer.
- **Domain Layer (Business Logic):** This is the core of the system, containing all the business rules and processes. It's organized into logical packages like Orders, Transfers, and Distribution, which makes the application's logic modular and easy to manage. This layer is completely independent of the UI.
- **Technical Services Layer (Infrastructure):** This layer provides essential technical functions to support the application. It handles all external concerns, including:
    - Persistence: Manages all communication with the Mongo Database.
    - Logging: Records application events and errors.
    - Identity & Access Management: Controls user authentication and permissions.



5.2 System Architecture and Design Philosophy

The system's architecture is founded on modern patterns that prioritize scalability, maintainability, and robust data management. The core philosophy was to build a decoupled system that leverages the strengths of its chosen technologies, rather than developing complex, custom computational algorithms.

Core Architectural Patterns

The system's foundation is built upon two key patterns: a REST API for client-server communication and the MVVM (Model-View-ViewModel) pattern for the client application.

1. REST API (Backend Communication)

A REST API was used to decouple the C# frontend from the Python backend, creating a clear boundary between the client and server. This was a strategic choice for several reasons:

- **Scalability and Flexibility:** The stateless nature of the REST API allows the backend to be scaled independently to handle increased load without affecting the client. It also provides a standardized interface, enabling future applications (e.g., a web dashboard or mobile app) to connect to the same backend logic.
- **Independent Development:** By defining a clear API contract, frontend and backend development can proceed in parallel. This separation allows each component to be updated and deployed independently, increasing development speed and simplifying maintenance.

2. MVVM Pattern (C# Frontend)

The MVVM pattern was implemented in the C# client application to create a clean separation between the user interface (UI) and the underlying logic. This directly contributes to the system's quality and longevity:

- **Enhanced Testability:** MVVM isolates the application's presentation logic into the ViewModel, which is independent of the visual UI elements. This allows for robust, automated unit testing of UI behaviour, ensuring the application is reliable.
- **Improved Maintainability and UI Efficiency**: By separating the system into a Model, View, and ViewModel, the codebase becomes highly modular and easier to manage. The pattern also leverages powerful data binding, which automatically synchronizes data and simplifies the code required to display real-time information like live inventory levels.

5.3 Approach to Algorithmic Complexity

This architectural choice directly influences the system's approach to algorithms. The design deliberately avoids custom computational algorithms because the system's primary role is process automation, not complex data calculation.

- **A Focus on Business Workflows (CRUD):** The application's core purpose is to manage business entities through standard data operations (Create, Read, Update, Delete). The logic involved is procedural—a sequence of defined business rules—rather than computationally algorithmic.
- **Delegation to High-Performance Tools:** Instead of re-implementing logic for tasks like searching, sorting, and data aggregation, this complexity is efficiently handled by the MongoDB database. MongoDB's internal query engine and aggregation framework contain highly optimized algorithms, performing these operations with far greater speed and efficiency than custom application-level code could achieve.

6. Implementation

6.0 User Interface implementation

The user interface was implemented using WinUI 3 and the C# language, with a strong emphasis on the Model-View-ViewModel (MVVM) design pattern to ensure a clean separation between the UI's appearance, its data, and its logic.

The entire layout and structure of the UI are defined in .xaml files. The application is architected using MVVM to manage complexity e.g. Delivery.cs, StockTake.cs represent the application's data structures(Model), the .xaml files (DeliveryPage.xaml, StockTakeForm.xaml) define what the user sees. DeliveryViewModel.cs, StockTakeViewModel.cs) act as the bridge. They fetch data from the Python API and expose it to the View, handling the application's presentation logic.

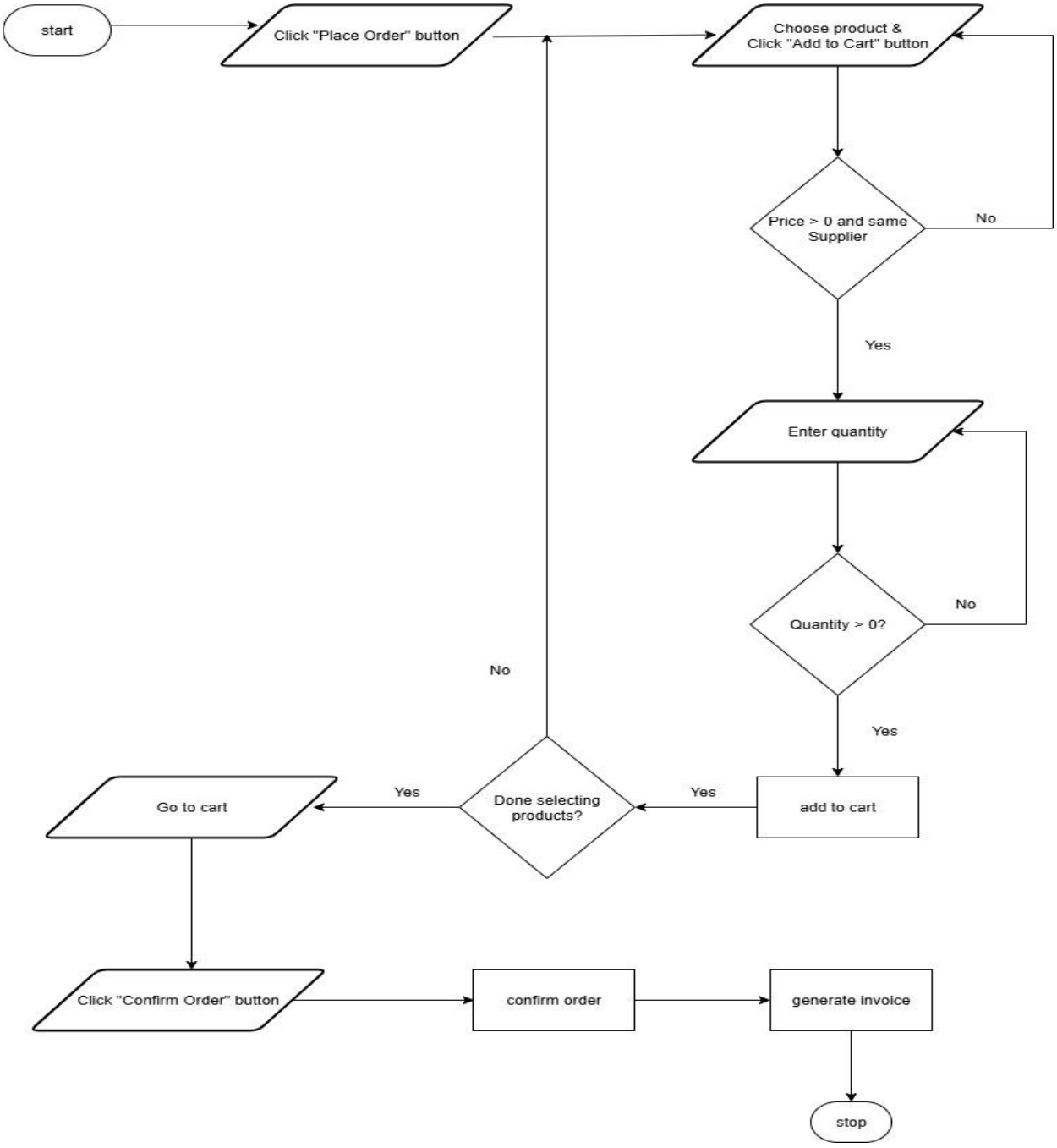
The application uses SfDataGrid control from Syncfusion to display lists of deliveries and stock takes. This was implemented by binding the grid's ItemsSource property to an ObservableCollection in the corresponding ViewModel. This allows the grid to support features like sorting, filtering, and grouping. Data entry forms, like DeliveryEntryForm.xaml and StockTakeForm.xaml, are built with a combination of standard controls (TextBox, Button) and advanced controls like SfDatePicker and SfComboBox for a better user experience. The forms use data binding to link UI elements directly to properties in the data models.

To keep the user informed, the application uses an InfoBar control. The ShowInfoBar method in the code-behind files can be called to display messages for success, warnings, or errors at the top of the page, which automatically dismisses after a few seconds. For critical actions, such as navigating away from a form with unsaved changes, the UI implements confirmation dialogs using ContentDialog. This prevents accidental data loss by prompting the user to confirm their action, improving the application's robustness.

6.1 Most significant methods of classes

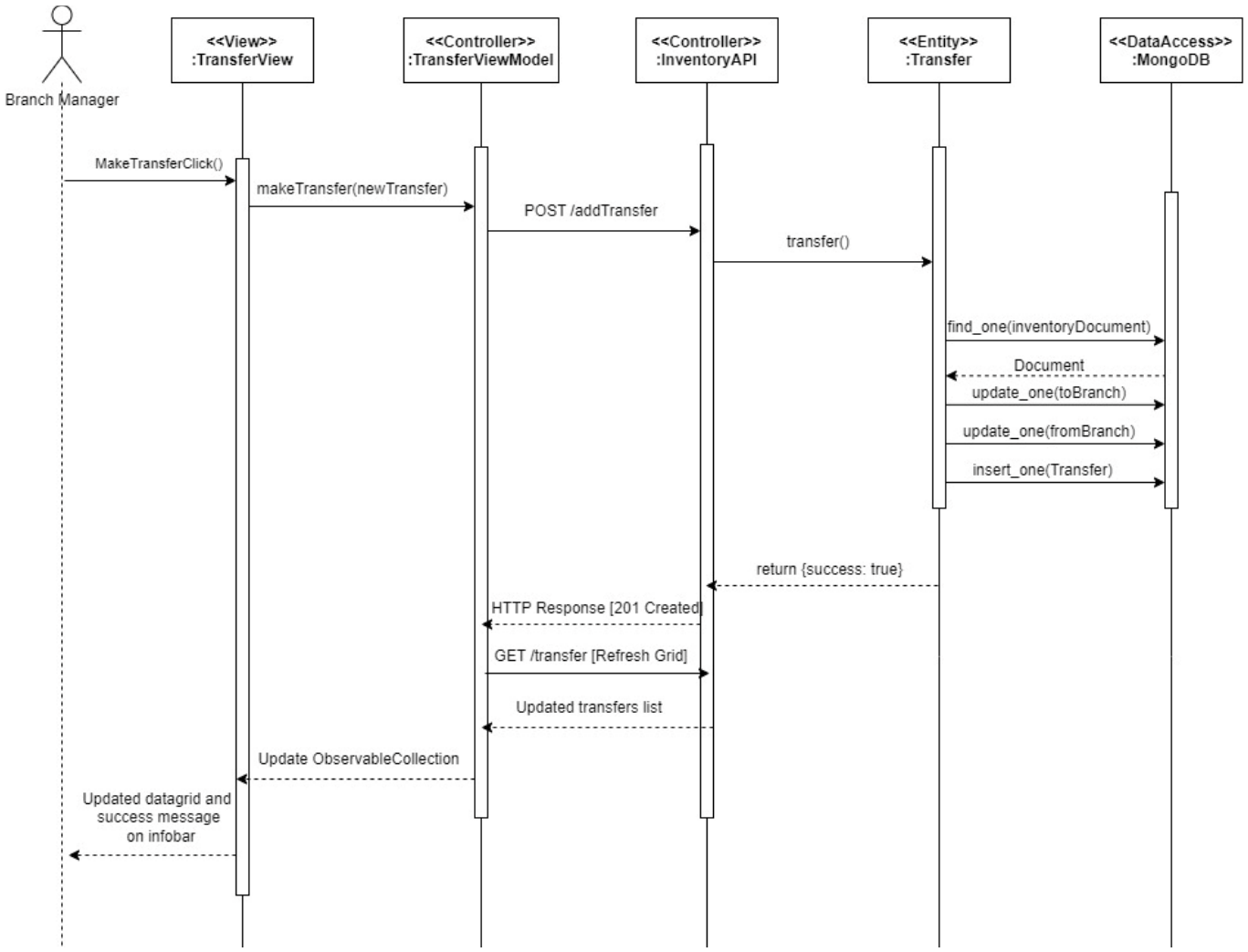
6.1.1 Orders

In the **Order** class, the steps involved in placing an order are outlined. This class allows a user to select products from a chosen supplier and, once all products have been added, provides the option to generate an invoice, email the supplier, or send an SMS. The sequence of steps executed by this class is illustrate in the diagram below.


start
Click "Place Order" button
Choose product &
Click "Add to Cart" button
Price > 0 and same
Supplier
No
Yes
Enter quantity
Quantity > 0?
No
Yes
No
add to cart
Yes
Done selecting
products?
Yes
Go to cart
Click "Confirm Order" button
confirm order
generate invoice
stop

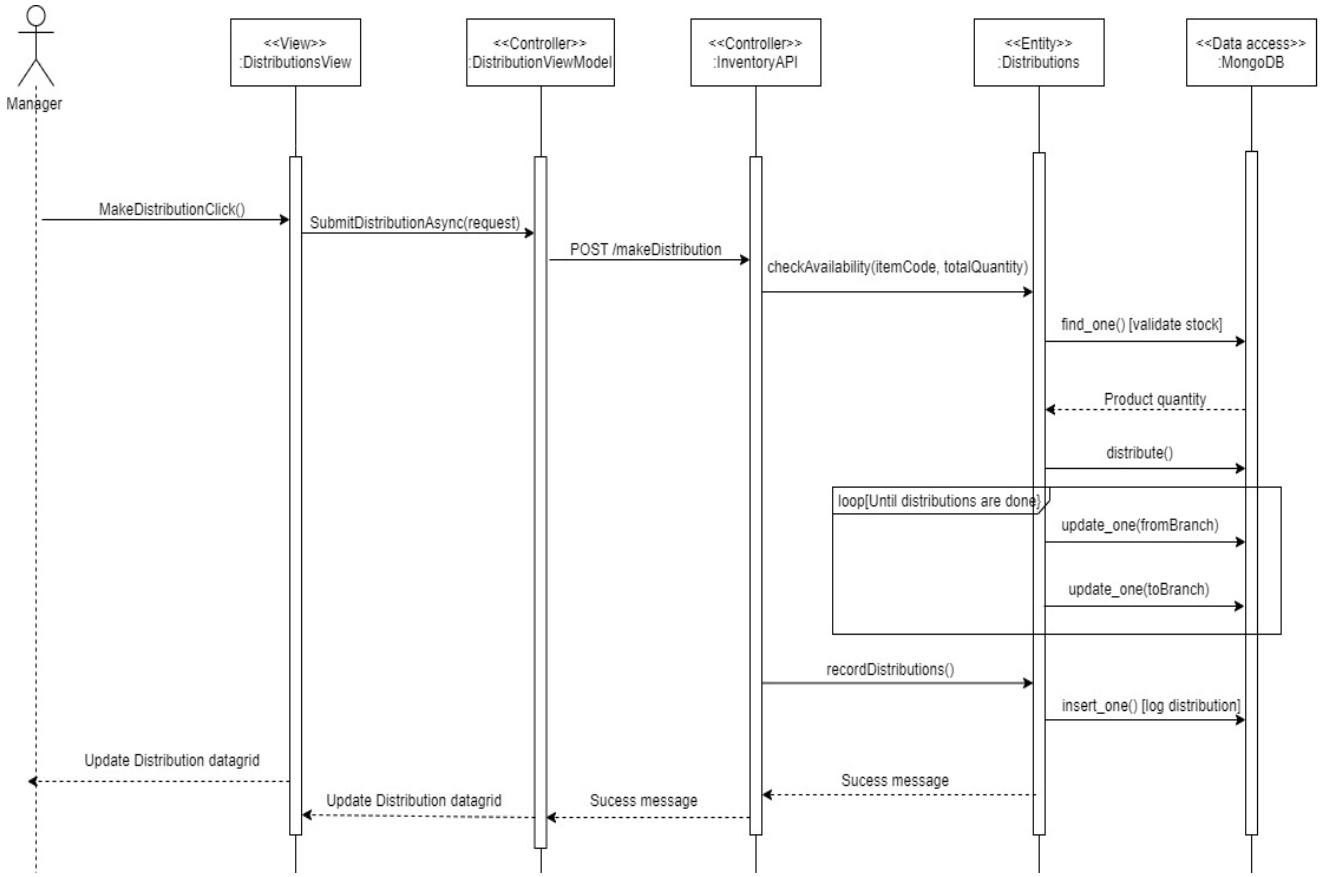
6.1.2 Transfers

In the **Transfer** class, the steps involved in processing a stock transfer are outlined, whether the transfer is from the main branch to another branch or between branches. The sequence of steps executed by this class is illustrated in the sequence diagram below.



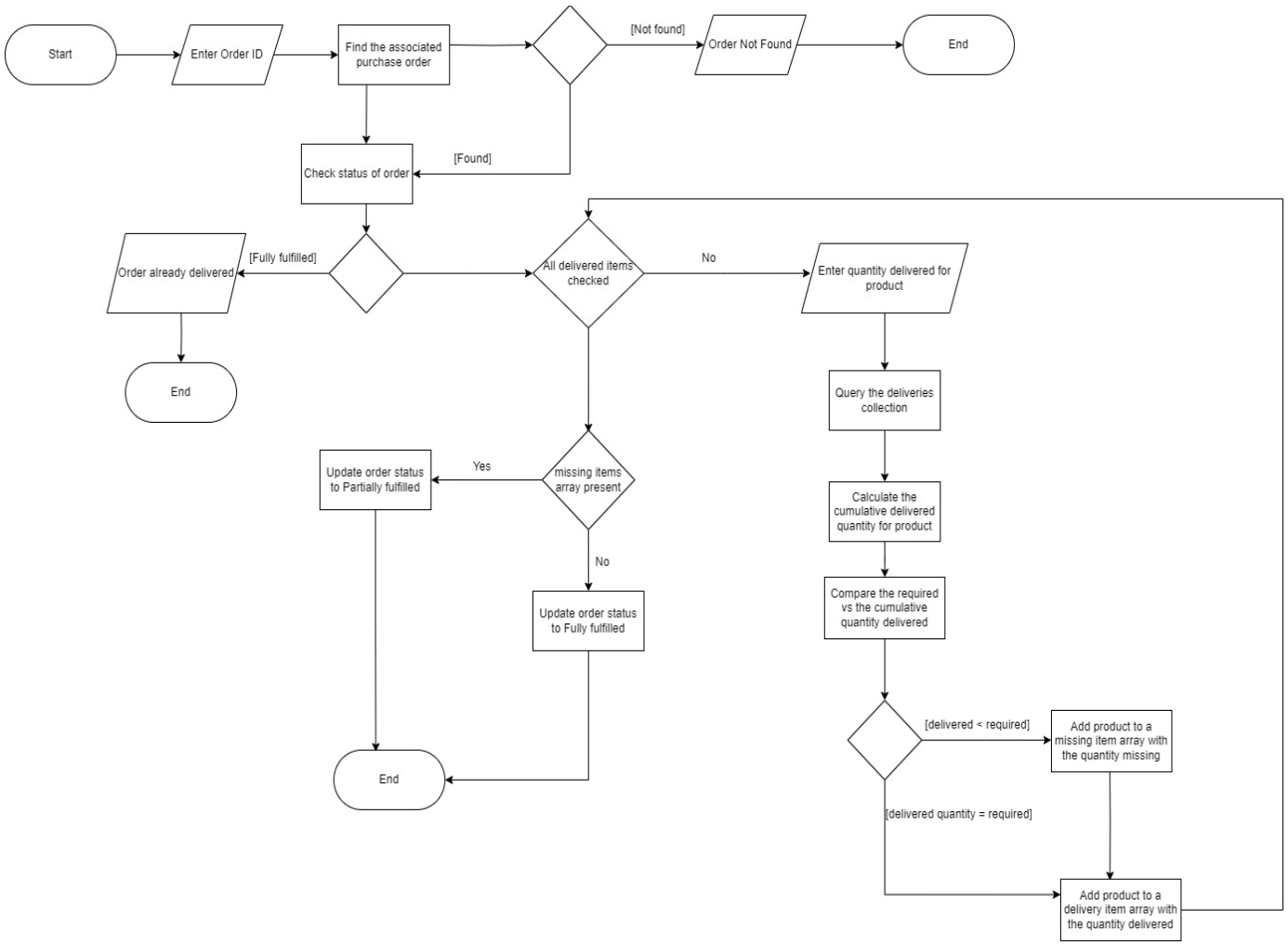
6.1.3 Distributions

In the **Distribution** class, the steps involved in distributing stock from the main branch to other branches are outlined. This class ensures that the quantities allocated are sufficient to meet the needs of all selected branches, while also maintaining enough stock in the main branch to satisfy its own demand. The sequence of steps executed by this class is illustrated in the sequence diagram below.



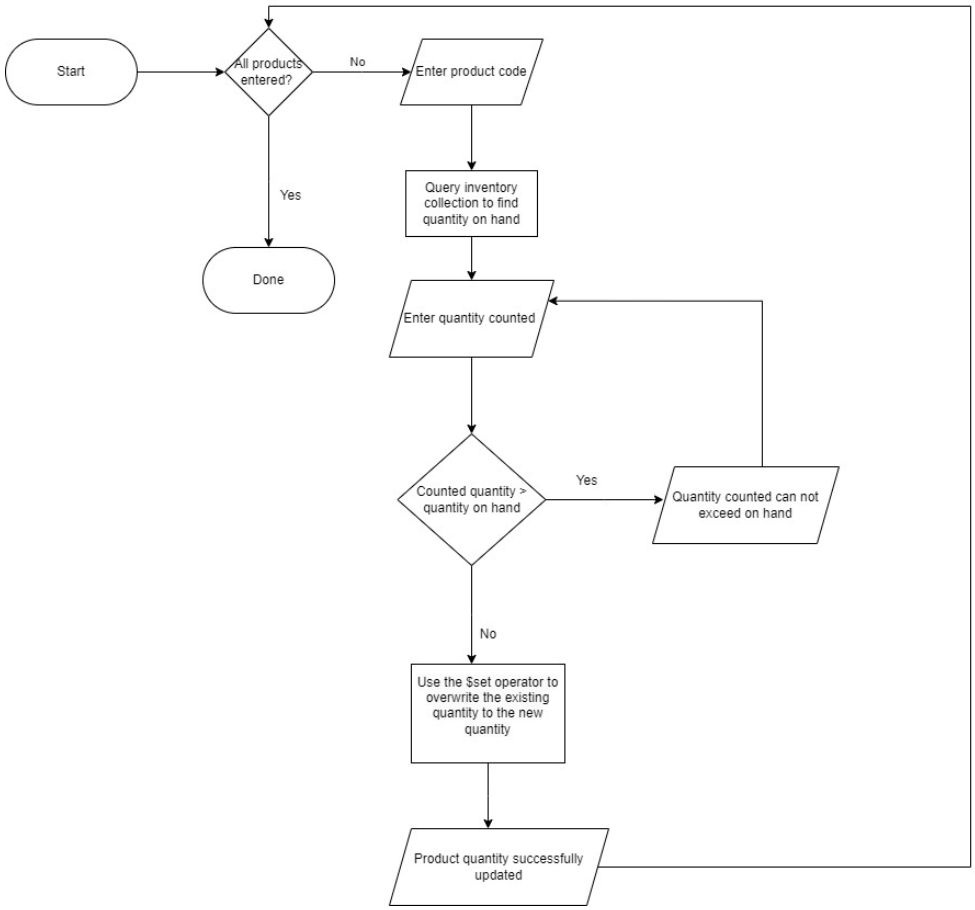
6.1.4 Delivery

In the **Delivery** class, an important method is update_order_status. This method updates an order's status to either *Partially Fulfilled* or *Fully Fulfilled* once a new delivery has been recorded. The sequence of steps executed by this method is illustrated in the flow chart below.



6.1.5 Stock Take

In the **StockTake** class, an important method is update_stock_levels. This method is responsible for adjusting the quantity of a product in the inventory whenever a stock take is performed. Its purpose is to ensure that, after each stock take, the database remains consistent with the actual quantities on hand. The sequence of steps executed by this method is illustrated in the flow chart below.



6.2 Special programming techniques or libraries used

FastAPI was used with asynchronous endpoints (async def) to enable non-blocking I/O operations, allowing the server to continue processing other requests while waiting for external operations. This approach improves overall performance and scalability. Pydantic was also employed to define BaseModel classes such as DeliveryData and StockTakeModel, ensuring that data received by the API endpoints automatically conforms to the required structure and data types. Invalid inputs are rejected with clear error messages, for example when a string is provided where an integer is expected.

7. Program Validation and Verification

7.0 System Testing and Quality Management

To ensure the reliability, correctness, and usability of the BabyBargains stock management system, a multi-layered Quality Management Plan was designed and executed. The core objective of this plan was to systematically validate the system at every level of its architecture, from individual code components to the fully integrated application. This rigorous approach confirms that the system not only meets all functional requirements but is also robust, secure, and performs efficiently under expected workloads.

7.1 Software Testing Plan

Our testing strategy was divided into four distinct phases, each targeting a different aspect of the system. This layered approach ensures that fundamental components are verified before being tested as part of a larger whole, which is a highly effective method for identifying and isolating defects early in the development lifecycle.

Table 1: *Summary Testing Plan.*

Process	Technique
1. Class Testing: test methods and state behaviour of classes	Random, Partition and White-Box Tests
2. Integration Testing: test the interaction of sets of classes	Random and Behavioural Testing
3. Validation Testing: test whether customer requirements are satisfied	Use-case based black box and Acceptance tests
4. System Testing: test the behaviour of the system as part of a larger environment	Recovery, security, stress and performance tests

Phase 1: **Class Testing:** The first phase focused on the smallest units of the system: individual classes and their methods. Using **White-Box Testing**, we examined the internal logic to ensure every path and condition was executed correctly. **Partition Testing** was used to divide inputs into equivalence classes (e.g., positive vs. negative quantities) to ensure each was handled correctly.

Phase 2: Integration Testing: This phase tested the interaction between components, with a key focus on the communication between the C# client and the Python REST API. **Behavioural Testing** was employed to simulate user actions and verify that API calls were constructed and handled correctly.

Phase 3: Validation Testing: The goal of this phase was to validate that the software met the business requirements. We used **Use-Case** Based Black **Box Testing**, where test cases were designed directly from the project requirements, treating the system as a "black box" to focus only on inputs and expected outputs.

Phase 4: System Testing: The final phase tested the application, including **Security Testing** (attempting unauthorized access), and **Recovery Testing** (ensuring the system could handle server restarts).

7.2 Validation of Program Correctness

To validate the program's correctness, a multi-faceted approach was taken, combining extensive automated backend testing with manual, use-case-driven validation of the frontend application.

The primary validation steps were:

Automated Unit & Integration Testing: A comprehensive suite of automated tests was developed for the Python backend using pytest. These tests verified each piece of business logic in isolation by "mocking" database interactions. Further integration tests were run against a temporary, live test database to validate the full workflow from API endpoint to database update.

Simulated User Acceptance Testing (UAT): We conducted rigorous internal UAT by simulating user roles. Team members, acting as branch managers, followed core user stories (e.g., performing transfers,

recording deliveries, conducting stocktakes). These manual tests were crucial for validating both the logical correctness and the user experience of the C# application.

7.3 Test Data and Results

For both automated and manual tests, specific data sets were chosen to cover three critical conditions: normal functioning, extreme boundary cases, and invalid data. This strategy ensures the system works in daily use, handles limits correctly, and is resilient to user error.

Table 2: *Summary of tests carried out. A table caption goes above the table.*

Data Set and reason for its choice	Normal Functioning	Extreme boundary cases	Invalid Data (program should not crash)
Branch Management (test_branch.py) Chosen to test core CRUD functionality for branches.	Passed	Passed	Passed
Inter-Branch Transfer (test_transfers.py) Chosen to test a key business process involving multiple data validations.	Passed	Passed	Passed
Stock Distribution (test_distribution.py) Chosen to validate inventory checks from the main branch and multi-branch updates.	Passed	Passed	Passed
User Authentication (test_user.py) Chosen to test the complete login and 2FA verification security workflow.	Passed	Passed	Passed
Delivery & Order Fulfilment (unit_test_delivery.py, intergration_test_delivery.py) Chosen to test partial vs. full delivery logic and its impact on order status.	Passed	Passed	Passed
Stocktake & Inventory Correction (unit_test_stocktake.py, intergration_test_stocktake.py) Chosen to test the primary mechanism for ensuring data accuracy in branch inventory.	Passed	Passed	Passed
API Endpoint Integration (test_api.py, etc.) Chosen to test the interaction between the API layer and the underlying business logic.	Passed	Passed	Passed
CSV Data Upload (test_product.py, etc.) Chosen to test bulk data import, including header and data validation.	Passed	Passed	Passed
User Story Validation (delivery_validation_test.py) Chosen to confirm that key acceptance criteria for user stories are met.	Passed	Passed	Passed

7.4 Discussion of Test Results

Testing confirmed that the Baby Bargains stock management system is both robust and reliable. All modules passed tests for normal operation, extreme boundary conditions, and invalid data handling, showing that the system meets its functional requirements.

Core processes such as branch management, transfers, distribution, deliveries, and stocktakes behaved correctly under expected and edge-case conditions. Invalid inputs, including malformed CSV files or negative quantities, were handled gracefully with error messages, ensuring the system did not crash and guiding users toward correction.

Integration tests verified smooth communication between the C# frontend and the Python REST API, while security tests confirmed that login and two-factor authentication blocked unauthorized access. User story validation further showed that the system supports real-world workflows for managers and staff.

Overall, results demonstrate the system's correctness, resilience, and readiness for operational deployment.

8. Conclusion

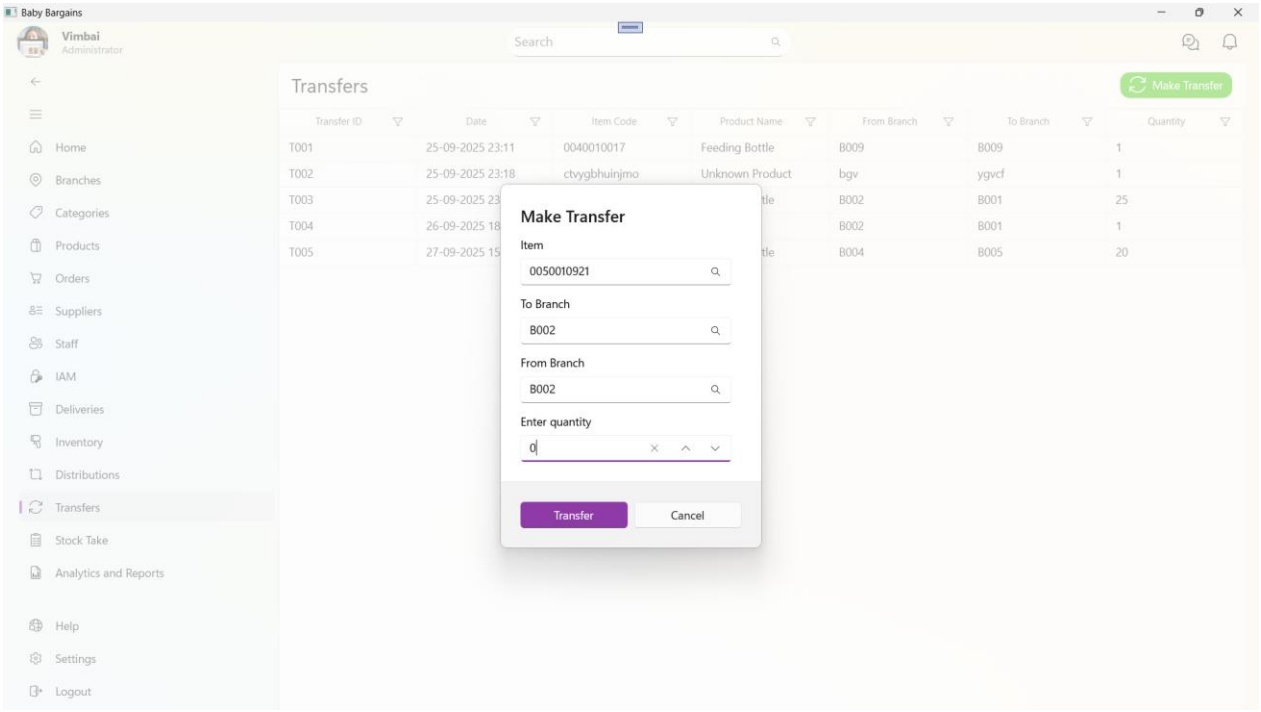
The BabyBargains stock management system successfully achieved its goal of replacing the manual paper-based process with a secure, efficient, and modular electronic solution. Built on a layered architecture, it supports all key operations including ordering, deliveries, stock distribution, stocktaking, and inter-branch transfers while enforcing role-based access for accountability. Testing confirmed that the system is correct, reliable, and resilient, with all modules passing normal, boundary, and invalid-input scenarios. Integration between the C# frontend and Python backend was seamless, and security tests validated strong protection against unauthorized access. Overall, the project delivered a comprehensive inventory management platform that is both technically sound and practically aligned with real business needs.

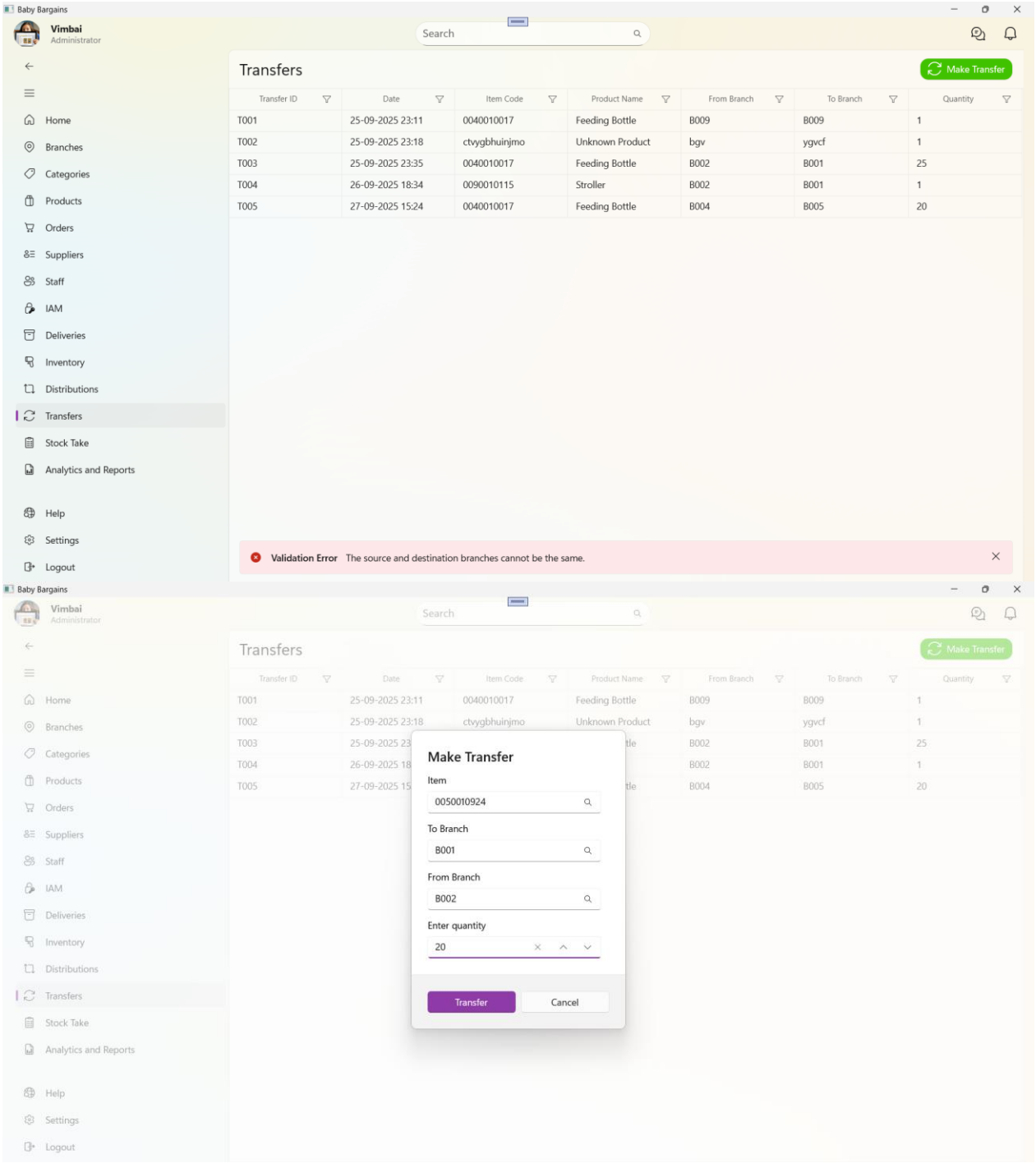
9. AI Clause

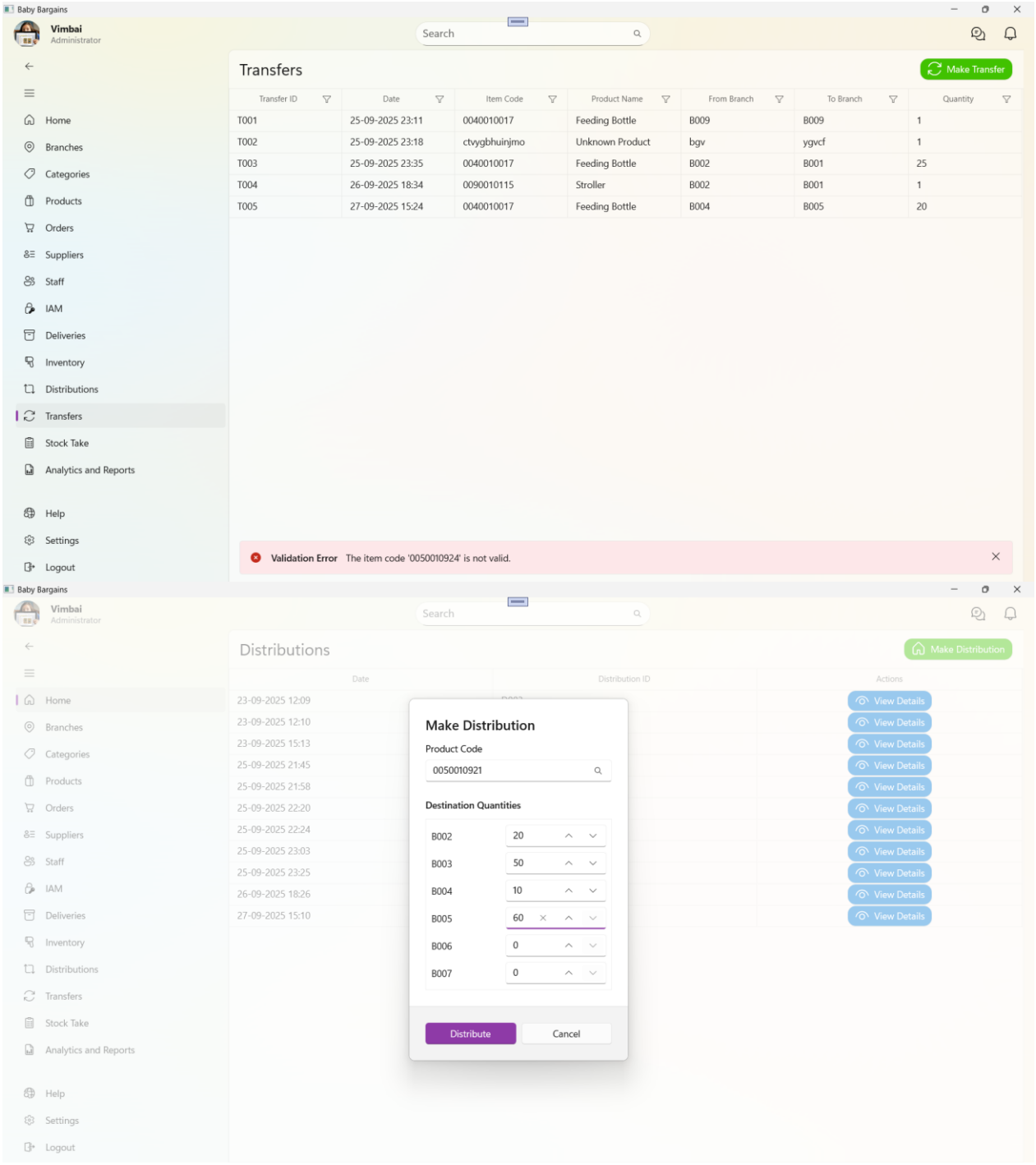
Artificial Intelligence (AI) tools were used during the development of this project to support and enhance the implementation process. Specifically, AI was employed to assist with debugging and compilation issues, and to provide guidance on using WinUI and XAML for the frontend, as this was our first time working with these technologies for desktop development. Additionally, AI was used to transform our wireframes into mock-up images, helping us to visualize and refine the frontend design. At all times, AI served as a supplementary tool for problem-solving and design support, while all final code, implementation decisions, and testing were carried out by the project team.

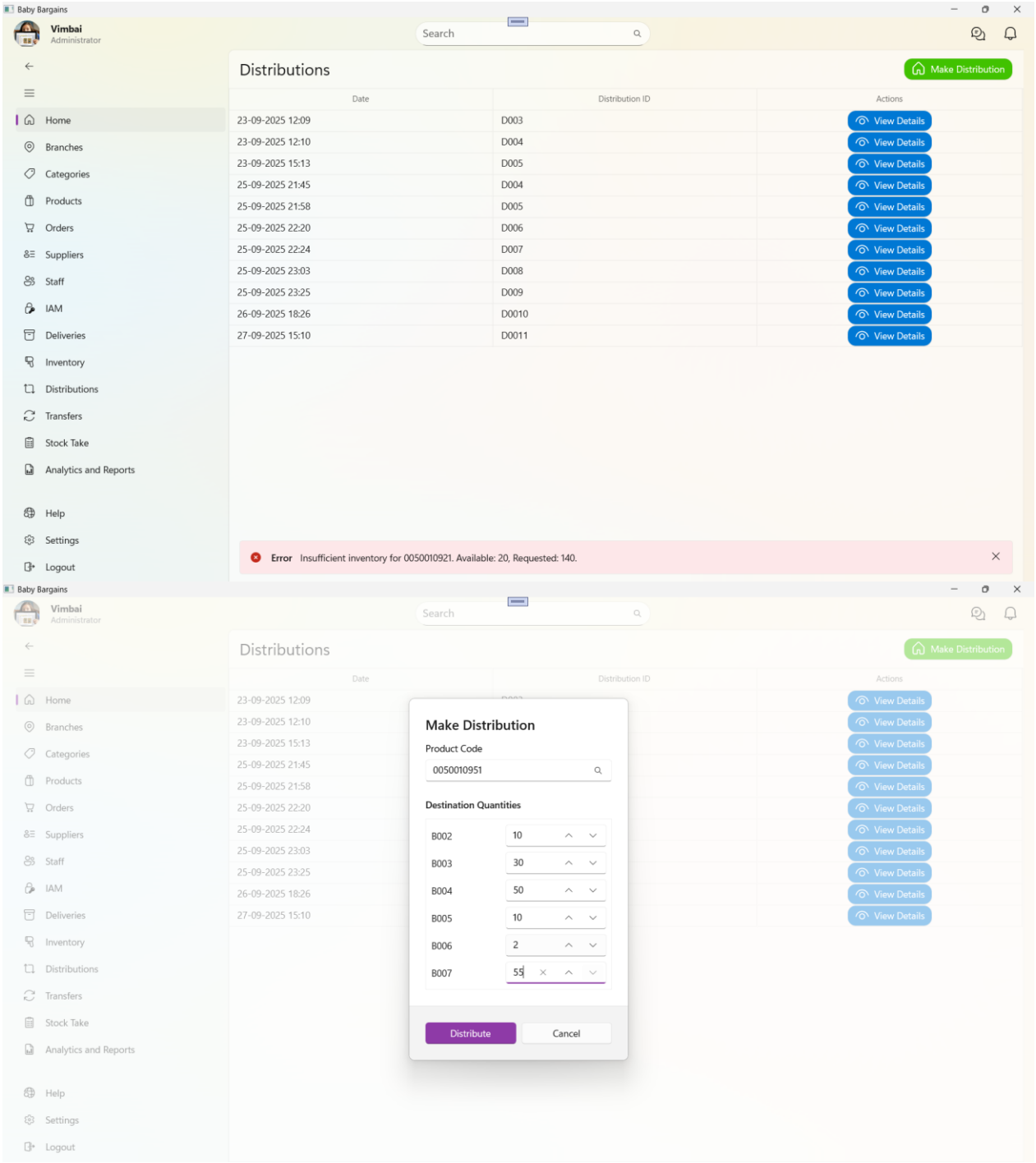
Appendix A — Evidence of testing

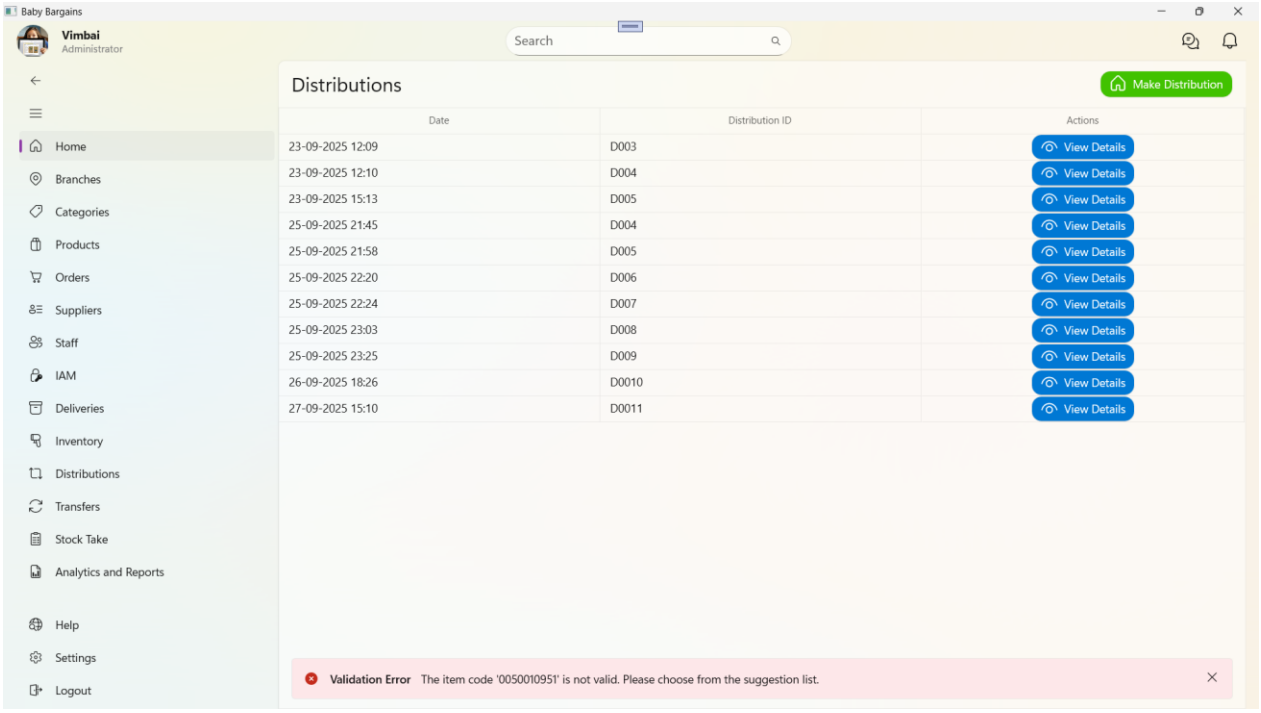
We placed our test files in the **tests** folder. For validation and verification, we applied both black-box testing (used for transfers and distribution) and acceptance testing (applied to the remaining components). The black-box testing results are shown below, while the acceptance tests are implemented in **delivery_validation_test.py**.

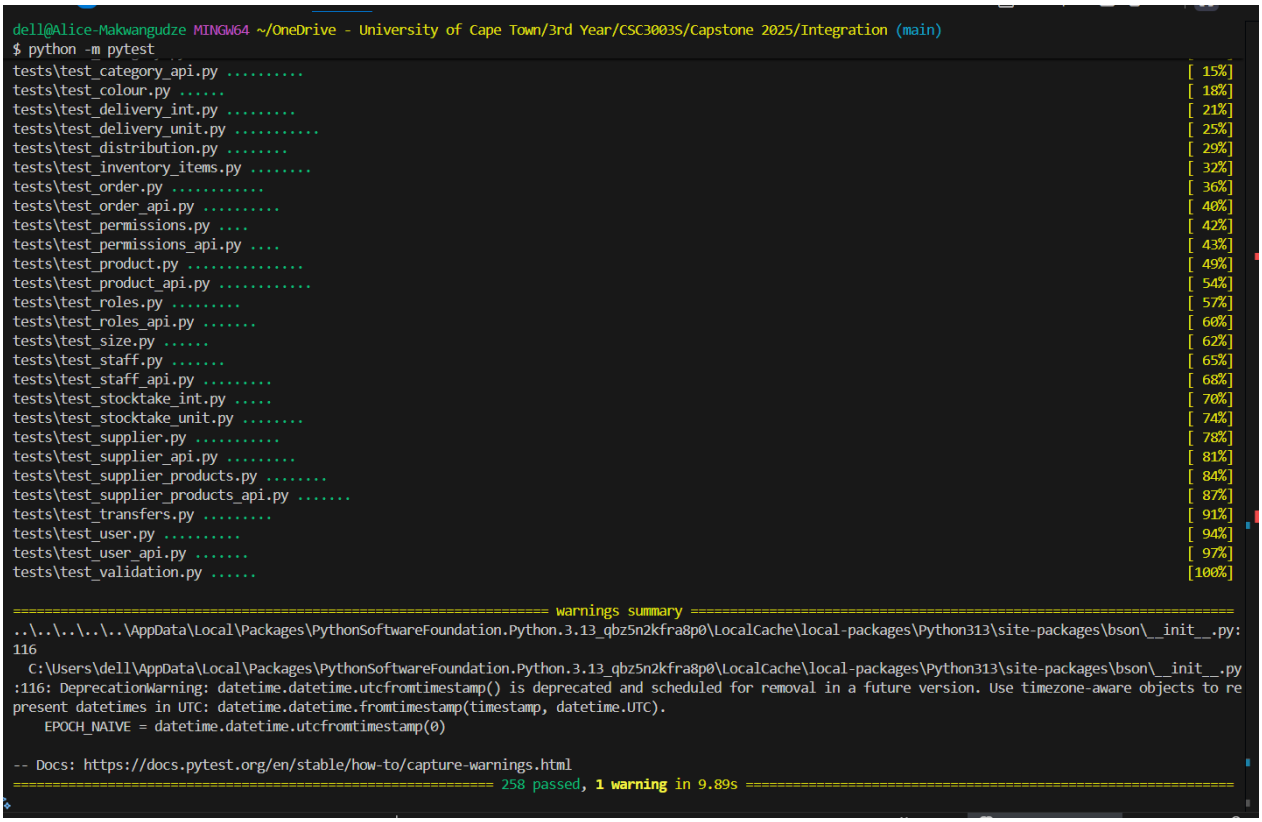



Baby Bargains
Vimbai
Administrator
Search
Transfers
Make Transfer
Transfer ID
Date
Item Code
Product Name
From Branch
To Branch
Quantity
T001 25-09-2025 23:11 0040010017 Feeding Bottle B009 B009 1
T002 25-09-2025 23:18 ctvygbhuinjmo Unknown Product bgv ygvcf 1
T003 25-09-2025 23:35 0040010017 Feeding Bottle B002 B001 25
T004 26-09-2025 18:34 0090010115 Stroller B002 B001 1
T005 27-09-2025 15:24 0040010017 Feeding Bottle B004 B005 20
Home
Branches
Categories
Products
Orders
Suppliers
Staff
IAM
Deliveries
Inventory
Distributions
Transfers
Stock Take
Analytics and Reports
Help
Settings
Logout
Validation Error The source and destination branches cannot be the same.
Make Transfer
Item
0050010924
To Branch
B001
From Branch
B002
Enter quantity
20
Transfer
Cancel


Baby Bargains
Vimbai
Administrator
Search
Transfers
Make Transfer
Transfer ID
Date
Item Code
Product Name
From Branch
To Branch
Quantity
T001 25-09-2025 23:11 0040010017 Feeding Bottle B009 B009 1
T002 25-09-2025 23:18 ctvygbhuinjmo Unknown Product bgv ygvcf 1
T003 25-09-2025 23:35 0040010017 Feeding Bottle B002 B001 25
T004 26-09-2025 18:34 0090010115 Stroller B002 B001 1
T005 27-09-2025 15:24 0040010017 Feeding Bottle B004 B005 20
Home
Branches
Categories
Products
Orders
Suppliers
Staff
IAM
Deliveries
Inventory
Distributions
Transfers
Stock Take
Analytics and Reports
Help
Settings
Logout
Validation Error The item code '0050010924' is not valid.
Distributions
Make Distribution
Date
Distribution ID
Actions
23-09-2025 12:09
23-09-2025 12:10
23-09-2025 15:13
25-09-2025 21:45
25-09-2025 21:58
25-09-2025 22:20
25-09-2025 22:24
25-09-2025 23:03
25-09-2025 23:25
26-09-2025 18:26
27-09-2025 15:10
View Details
Make Distribution
Product Code
0050010921
Destination Quantities
B002 20
B003 50
B004 10
B005 60
B006 0
B007 0
Distribute
Cancel


Baby Bargains
Vimbai
Administrator
Search
Distributions
Make Distribution
Date
Distribution ID
Actions
23-09-2025 12:09 D003 View Details
23-09-2025 12:10 D004 View Details
23-09-2025 15:13 D005 View Details
25-09-2025 21:45 D004 View Details
25-09-2025 21:58 D005 View Details
25-09-2025 22:20 D006 View Details
25-09-2025 22:24 D007 View Details
25-09-2025 23:03 D008 View Details
25-09-2025 23:25 D009 View Details
26-09-2025 18:26 D0010 View Details
27-09-2025 15:10 D0011 View Details
Home
Branches
Categories
Products
Orders
Suppliers
Staff
IAM
Deliveries
Inventory
Distributions
Transfers
Stock Take
Analytics and Reports
Help
Settings
Logout
Error Insufficient inventory for 0050010921. Available: 20, Requested: 140.
Make Distribution
Product Code
0050010951
Destination Quantities
B002 10
B003 30
B004 50
B005 10
B006 2
B007 55
Distribute
Cancel





Appendix B— User Manual and Code

We have attached the user manual and the code to the submitted files